
Implementing Security Measures

by:

Arab Hammad

1 Report Overview

1.1 Executive Summary

This report constitutes the formal remediation deliverable for Week 2 of the Hackazone security engagement. Following the penetration test conducted in Week 1, five primary vulnerability classes were identified: Local File Inclusion (LFI), Session Hijacking, Brute-Force / Username Enumeration, Insecure File Upload, and Cross-Site Scripting (Stored & Reflected). Additionally, supporting weaknesses were found in the areas of plain-text password storage, absent security headers, CSRF token bypass, and open redirect.

This document provides production-ready Node.js / Express remediation code for each vulnerability, organized into five implementation pillars: Input Validation & Sanitization, Password Hashing with bcrypt, JWT-based Authentication, Security Headers via Helmet.js, and Vulnerability-Specific Code Fixes. Every code sample is commented to explain the security logic behind it

2 Input Validation & Sanitization

2.1 Security Rationale

The Core Problem:

The vulnerabilities identified in Week 1 (XSS, SQLi, LFI) share one root cause: Implicit Trust. The application processed user input without proper validation or sanitization.

The Solution:

Whitelisting vs. Blacklisting

- Blacklisting (Weak): Trying to block "known bad" characters (e.g., <script>). This is unreliable because attackers always find new ways to bypass filters.
- Whitelisting (Strong): Only allowing "known good" data that matches specific criteria (e.g., only letters and numbers). Anything else is Rejected by Default.

Implementation Principle:

In this remediation, we implemented a "Zero Trust" input policy using the validator library. By enforcing strict data types, lengths, and formats, we neutralize attack vectors before they reach the server's business logic.

2.2 Reusable Validation Middleware

```
const validator = require('validator');
```

```
=====
1. Helper Function: Sanitise Text
=====
```

Purpose:

- Remove dangerous characters
- Prevent XSS attacks
- Clean user-generated content

Used for:

- comments
- usernames
- titles
- free-text input

```
const sanitiseText = (str) => {

  if (typeof str !== 'string') {
    return "";
  }
  str = str.trim();
  str = validator.stripLow(str);
  str = validator.escape(str);
  return str;
};
```

2. Registration Validator Middleware

Validates:

- Email format
- Username format
- Password strength

```
exports.validateRegistration = (req, res, next) => {
```

```
  const { email, password, username } = req.body;
  if (!email || !validator.isEmail(email)) {
    return res.status(400).json({error: 'Invalid email address.'
  });
}
```

```
  if (
    !username || !validator.isAlphanumeric( username.replace(/_/g, 'a')))
  {
    return res.status(400).json({error: 'Username must contain only letters, numbers, and underscores.'
  });
}
```

```
  if (!validator.isLength(username, {
    min: 3,max: 20}))
  {
    return res.status(400).json({error: 'Username must be between 3 and 20 characters.'
  });}
```

```
  if (!password ||!validator.isStrongPassword(password, {
    minLength: 8,
    minLowercase: 1,
    minUppercase: 1,
    minNumbers: 1,
    minSymbols: 1,
  })
  ) {
    return res.status(400).json({error:'Password must contain uppercase, lowercase, number, and symbol.'});}
```

```
req.body.username = sanitiseText(username);

req.body.email = validator.normalizeEmail(email);
next();
};
```

3. Login Validator Middleware

Purpose:

- Validate login input
- Prevent user enumeration

```
exports.validateLogin = (req, res, next) => {

  const { email, password } = req.body;

  if (!email || !validator.isEmail(email) || !password) {
    return res.status(400).json({error: 'Invalid credentials.'
  });
}

req.body.email = validator.normalizeEmail(email);
next();
};
```

4. Comment / FAQ Validator Middleware

Purpose:

- Prevent Stored XSS
- Validate user-generated content

```
exports.validateComment = (req, res, next) => {

  const { title, body } = req.body;

  if (!title || !validator.isLength(title, {min: 1, max: 120})
  ) {
    return res.status(400).json({error: 'Title must be between 1 and 120 characters.'
  });
}

req.body.title = sanitiseText(title);
```

```
req.body.body = sanitiseText(body || "");
next();
};
```

3 Password Hashing with bcrypt

3.1 Why bcrypt?

The Problem:

Plain-Text Storage Storing passwords in plain text is a critical risk. If the database is compromised (e.g., via SQL Injection), every user's credentials are immediately exposed. This leads to Credential Stuffing, where attackers reuse these passwords to hijack the users' other accounts (email, social media, etc.).

The Solution:

We implemented bcrypt because it is a "slow-by-design" adaptive hashing algorithm. Unlike fast algorithms (MD5, SHA-256) that can be cracked in seconds, bcrypt features a Cost Factor that intentionally slows down the process.

- **Brute-Force Protection:** By making each hash take ~250ms, it becomes computationally impossible for attackers to run large-scale offline attacks.
- **One-Way Security:** bcrypt hashes are irreversible. Even if an attacker or a database administrator gains access, they can never recover the original passwords.

3.2 Registration: Hashing the Password

```
const bcrypt = require('bcrypt');
const User = require('../models/User');
```

1. Security Configuration

Cost factor controls hashing strength:

- Higher = more secure but slower
- OWASP recommends ≥ 10

```
const BCRYPT_COST = 12;
```

2. Register User

Flow:

1. Get user input (already validated by middleware)
2. Check if user exists
3. Hash password
4. Save user to database

```
exports.register = async (req, res) => {
```

```
  try {
    const { email, username, password } = req.body;
```

```
const existing = await User.findOne({ email });
if (existing) {
  return res.status(400).json({
    error: 'Registration failed. Please try again.'
  });
}
```

----- 3. Hash password (bcrypt) -----

bcrypt automatically:

- generates salt
- applies hashing
- embeds salt in final hash

```
const hashedPassword = await bcrypt.hash(password,BCRYPT_COST);
```

----- 4. Create user object -----

```
const user = new User({
  email,
  username,
  password: hashedPassword
});
```

----- 5. Save to database -----

```
await user.save();
```

----- 6. Success response -----

```
res.status(201).json({
  message: 'Account created.'
});
```

```
} catch (err) {
  console.error('[register]', err.message);
```

```
  res.status(500).json({
    error: 'Server error.'
  });
}
};
```


3.3 Login: Comparing with bcrypt.compare()

```
const bcrypt = require('bcrypt');
const User = require('../models/User');
```

1. Login Function

Flow:

1. Get user input
2. Find user in database
3. Secure password comparison (bcrypt)
4. Return JWT if authenticated

```
exports.login = async (req, res) => {
```

```
  try {
    const { email, password } = req.body;
```

```
    const user = await User.findOne({ email });
```

2. Timing Attack Protection

```
const dummyHash =
'$2b$12$invalidhashfortimingprotection000000000000000000000000';
```

```
const hashToCompare = user ? user.password : dummyHash;
```

4. Password comparison

```
const isMatch = await bcrypt.compare(password, hashToCompare);
```

5. Authentication check

```
if (!user || !isMatch) {
  return res.status(401).json({ error: 'Invalid credentials.' });
}
```

6. Generate authentication token

```
const token = generateToken(user);
```

```
return res.json({ token });
```

```

} catch (err) {

-----
Error handling
-----
console.error('[login]', err.message);

return res.status(500).json({
  error: 'Server error.'
});
}
};

```

4 Enhanced Authentication via JWT

4.1 Why JWT Replaces Session Cookies

The Problem:

Session Vulnerabilities The initial assessment of Hackazone revealed critical flaws in traditional session management (PHPSESSID), such as Session Hijacking and Session Fixation. Because the server struggled to invalidate session identifiers upon logout, attackers could persist in the system using stolen cookies.

The Solution:

Stateless Authentication (JWT) We replaced traditional cookies with JSON Web Tokens (JWT) to eliminate server-side session state.

- **Self-Contained:** The token carries the user's identity, so the server doesn't need to query the database for every request.
- **Cryptographic Integrity:** Every token is digitally signed. If an attacker attempts to modify the user ID or role within the token, the signature check will fail, and the request will be rejected.

Structure of the JWT:

1. **Header:** Defines the hashing algorithm (e.g., HS256).
2. **Payload:** Contains non-sensitive user claims (ID, Role, Expiration).
3. **Signature:** A secure hash created using a Secret Key to prevent tampering.

4.2 Token Generation

```

const jwt = require('jsonwebtoken');
const JWT_SECRET = process.env.JWT_SECRET;
const JWT_EXPIRES = '15m';
const REFRESH_EXP = '7d';

if (!JWT_SECRET) {
  throw new Error('JWT_SECRET environment variable is not set. Refusing to start.');
```

```
=====
```

1. Generate Access Token

```
=====
```

Purpose:

- Create signed token containing user identity
 - Used for authentication on each request
 - Stateless (no session stored on server)
- ```
=====
```

```
exports.generateToken = (user) => {
 return jwt.sign(
 { id: user._id , role: user.role },
 JWT_SECRET,
 {
 expiresIn: JWT_EXPIRES,
 issuer: 'hackazone-api',
 audience: 'hackazone-client'
 }
);
};
```

```
=====
```

## 2. Generate Refresh Token

```
=====
```

Purpose:

- Used to renew expired access tokens
  - Stored in HttpOnly cookie for security
- ```
=====
```

```
exports.generateRefreshToken = (user) => {
  return jwt.sign(
    {id: user._id},
    JWT_SECRET,
    {
      expiresIn: REFRESH_EXP,
      issuer: 'hackazone-api'
    });
};
```

4.3 Authentication Middleware (Protecting Private Routes)

```
const jwt = require('jsonwebtoken');
const { JWT_SECRET } = process.env;
```

```
=====
```

1. Authentication Middleware

```
=====
```

Purpose:

- Protect private routes
- Verify JWT token validity
- Attach user data to request

=====

```
module.exports = (req, res, next) => {

  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];

  if (!token) {
    return res.status(401).json({
      error: 'Authentication required.'
    });
  }

  try {
    const decoded = jwt.verify(
      token,
      JWT_SECRET,
      {
        issuer: 'hackazone-api',
        audience: 'hackazone-client'
      }
    );

    req.user = decoded;
    next();
  }
  catch (err) {
    const msg =
      err.name === 'TokenExpiredError'
        ? 'Session expired. Please log in again.'
        : 'Invalid token.';
    return res.status(401).json({ error: msg });
  }
}
```

=====

2. Role-Based Access Control (RBAC)

=====

Purpose:

- Restrict access based on user role
- Example: admin-only routes

=====

```
exports.requireRole = (role) => {

  return (req, res, next) => {
    if (req.user.role !== role) {
      return res.status(403).json({ error: 'Insufficient permissions.' });
    }
  }
}
```

```
next();});};
```

5 Security Headers with Helmet.js

```
const express = require('express');
const helmet = require('helmet');
```

```
const app = express();
```

1. Helmet Middleware Initialization

Purpose:

- Sets multiple HTTP security headers automatically
- Protects against client-side attacks (XSS, clickjacking, etc.)
- Must be applied before all routes

```
app.use(helmet({
```

1. Content Security Policy (CSP)

Purpose:

- Prevent Cross-Site Scripting (XSS)
- Controls what resources the browser is allowed to load

```
contentSecurityPolicy: {
  directives: {
    defaultSrc: ["'self'"],
    scriptSrc: ["'self'"],
    styleSrc: ["'self'", "'unsafe-inline'"],
    imgSrc: ["'self'", "data:"],
    connectSrc: ["'self'"],
    fontSrc: ["'self'"],
    objectSrc: ["'none'"],
    frameAncestors: ["'none'"],
    upgradeInsecureRequests: []
  }
},
```

2. HTTP Strict Transport Security (HSTS)

Purpose:

- Forces HTTPS connections
- Prevents SSL stripping attacks

```
hsts: {
  maxAge: 31536000,
```

```
includeSubDomains: true,  
preload: true  
},
```

3. MIME Sniffing Protection

Purpose:

- Prevents browser from guessing file types
- Stops malicious files disguised as images/scripts

```
noSniff: true,
```

4. Clickjacking Protection

Purpose:

- Prevents embedding site inside iframes
- Blocks UI redress attacks

```
frameguard: { action: 'deny' },
```

5. Hide Server Information

Purpose:

- Removes "X-Powered-By: Express"
- Reduces fingerprinting attacks

```
hidePoweredBy: true,
```

6. XSS Protection (Legacy)

Disabled because modern browsers rely on CSP instead

```
xssFilter: false  
));
```

2. Additional Security Middleware

Purpose:

- Limit request size
- Prevent DoS via large payloads

```
app.use(express.json({ limit: '10kb' }));
```